

Design and Simulation of an 8-Bit SAP-1 Computer Using Proteus

A Ben Eater-Inspired 8-Bit Computer Architecture

Ahmed Muhammad Karam

August 18, 2026

Contents

1	Introduction	3
1.1	The SAP-1 Architecture	3
1.2	SAP-1 Is Not Turing-Complete	4
1.3	Ben Eater's Turing-Complete SAP-1	4
2	Clock Module Design	6
3	A Register and B Register Design	8
4	Arithmetic Logic Unit (ALU) Design	10
5	Program Counter Design	12
6	RAM Module Design	14
6.1	Storage: Cascaded 74LS189 Static RAM	14
6.2	Data Selection and Synchronized Write Control	14
6.3	Memory Address Register (MAR)	15
7	Instruction Register Design	16
8	Output Module and Display Design	18
8.1	Output Register	18
8.2	Decimal Decoding: EEPROM Lookup Table	19
8.3	Choice of EEPROM: 27C256	19
8.4	Multiplexed Multi-Digit Display	20
8.5	Signed and Unsigned Display Modes	20
9	Control Unit Design	21
9.1	Step Counter	21
9.2	Microcode ROM Addressing	21
9.3	Byte-Select Addressing and Identical ROM Programming	22
9.4	Control Signal Buffering	22
9.5	Control Signal Reference Table	22
10	Reset Module Design	24

11 Instruction Set and Example Programs	26
11.1 The Extended Instruction Set	26
11.2 Program 1: Addition of Two Values	26
11.3 Program 2: Multiplication by Repeated Addition	27
11.4 Program 3: Division by Repeated Subtraction	27
11.5 Program 4: Data-Dependent Looping (Up/Down Counter)	28
11.6 Significance	28
11.7 Conclusion	29
References	30
Project Repository	30

Abstract

This project presents the design and simulation of an 8-bit computer built almost entirely from discrete 74-series logic in the Proteus simulation environment, based on the architectural principles of Ben Eater's SAP-1 computer[1]. The system incorporates a flexible clock module, shared 8-bit tri-state bus, general-purpose registers, an instruction register, memory address register, 16-byte RAM, 4-bit program counter, 8-bit arithmetic logic unit supporting addition and two's-complement subtraction, and a multiplexed 4-digit 7-segment display driven by an EEPROM lookup table. Control is implemented using a microprogrammed sequencer consisting of a step counter and two EEPROMs that generate the control signals required for instruction execution. The design extends the conventional SAP-1 architecture with a hardware flags register for carry and zero conditions and introduces conditional Jump-on-Carry and Jump-on-Zero instructions by incorporating the flag state into the microcode address space. These enhancements enable conditional branching and loop-based programs, including multiplication and division through repeated addition and subtraction. The completed system was verified using progressively complex test programs, during which several hardware faults, including bus contention during conditional jumps, were identified and resolved. The final implementation successfully executes arithmetic, memory, and branching operations, demonstrating a functional general-purpose computer constructed and verified almost entirely from discrete logic components.

1 Introduction

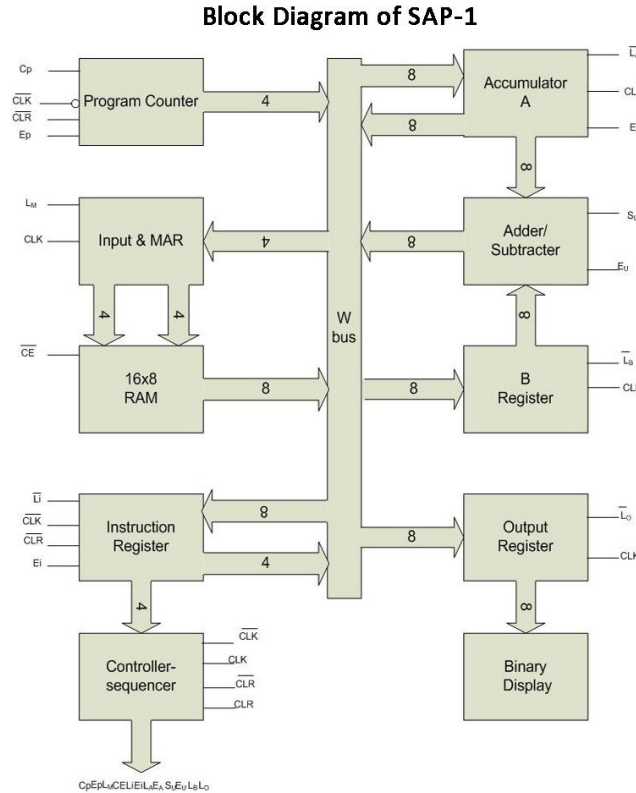


Figure 1

1.1 The SAP-1 Architecture

SAP-1 (Simple-As-Possible, One) is a foundational teaching architecture introduced by Albert Paul Malvino in Digital Computer Electronics[2] as the first in a three-stage progression (SAP-1, SAP-2, SAP-3) designed to introduce the core concepts of computer operation without the complexity of a commercial microprocessor. Despite its simplicity, SAP-1 embodies every fundamental subsystem found in a general-purpose computer: a bus-organized datapath, a program counter that sequences instruction fetches, a memory address register and random-access memory, an instruction register that decodes the operation to be performed, an arithmetic logic unit built from an accumulator and B register feeding a 2's-complement adder/subtractor, an output register driving a display, and a control unit that generates the timing and control signals coordinating all of the above.

The SAP-1 control unit operates on a fixed six-state machine cycle (T1 through T6), generated by a ring counter, split into a three-state fetch cycle (address, increment, memory states) common to every instruction, followed by a three-state execution cycle whose behavior depends on the decoded opcode. Malvino presents two equivalent ways of generating the resulting control word: a hardwired control matrix built from combinational logic gates directly decoding the instruction and timing-state bits, and an alternative microprogrammed implementation in which the control word is instead stored in ROM and addressed by the instruction and step. The textbook's na-

tive instruction set is deliberately minimal, consisting of LDA, ADD, SUB, OUT, and HLT, five instructions sufficient to fetch data from memory, accumulate a running sum or difference, and display the result.

1.2 SAP-1 Is Not Turing-Complete

This minimal instruction set has an important theoretical consequence: SAP-1, as originally specified, is not Turing-complete. Every instruction executes unconditionally and in strict program-counter order; there is no mechanism by which the outcome of a computation can alter the subsequent flow of execution. Consequently, SAP-1 cannot express loops, conditional branches, or any algorithm whose control flow depends on data, which rules out, among other things, multiplication, division, and any iterative process, regardless of how much memory or time is available.

1.3 Ben Eater's Turing-Complete SAP-1

The system designed and simulated in this project follows the architectural blueprint of Ben Eater's breadboard 8-bit computer, which implements the same SAP-1 core described above but extends it with the specific mechanism needed to overcome this limitation. This is a hardware flags register that captures the carry-out and zero-detect conditions from the ALU after every ADD or SUB operation, together with two new conditional instructions, Jump-on-Carry (JC) and Jump-on-Zero (JZ). Critically, this is implemented not through additional external decision logic, but by folding the flag bits directly into the address space of the microcode control ROM alongside the opcode and step counter, so that the same lookup-table mechanism generating every other control word also determines, transparently, whether a jump is taken. This closes a feedback loop between the ALU's output and the sequencer's own behavior, which is precisely the missing ingredient identified by the Church-Turing thesis as necessary for general computability: a conditional action, taken on the basis of previously computed data. With this addition, the resulting machine can express arbitrary loops and data-dependent branching, and is therefore Turing-complete, capable in principle of computing any function that is computable at all, memory permitting.

Table 1: Comparison Between the Textbook SAP-1 Architecture and the Developed Computer

Aspect	Textbook SAP-1	This Project	Enhancement
Instruction set	LDA, ADD, SUB, OUT, and HLT	NOP, LDA, STA, LDI, ADD, SUB, JMP, JC, JZ, OUT, and HLT	Expanded instruction set supporting immediate data, memory stores, and conditional branching
Memory operations	Primarily supports loading data from memory using LDA; arithmetic instructions operate on memory operands	Supports both memory load and store operations through LDA and STA, in addition to immediate loading through LDI	Provides more flexible data movement and program construction
Control flow	Strictly sequential; the program counter normally increments to the next instruction	Supports sequential execution, unconditional jumps through JMP, and conditional branches through JC and JZ	Enables loops, decision-making, and data-dependent program execution
Flags / status	No dedicated status flags	Carry and Zero flags are generated by the ALU and latched for subsequent instructions	Provides the status information required for conditional branching
Control unit	Control signals are generated from the opcode and timing step, with a separate address mechanism for variable instruction timing	Control signals are generated from the opcode, timing step, and flag state through a unified ROM-based lookup	Allows conditional instructions to directly influence microcode execution
Clocking	Basic clocking scheme for sequential operation and manual stepping	Automatic and manual clock modes with a dedicated halt-gating mechanism	Provides flexible program execution, debugging, and controlled single-step operation
Programmability	Limited instruction set suitable primarily for demonstrating basic CPU operation	Expanded instruction set capable of implementing arithmetic, memory manipulation, loops, and conditional logic	Significantly increases the range of executable programs
Turing-complete	No	Yes	Conditional branching and writable memory provide the fundamental mechanisms required for general computation

2 Clock Module Design

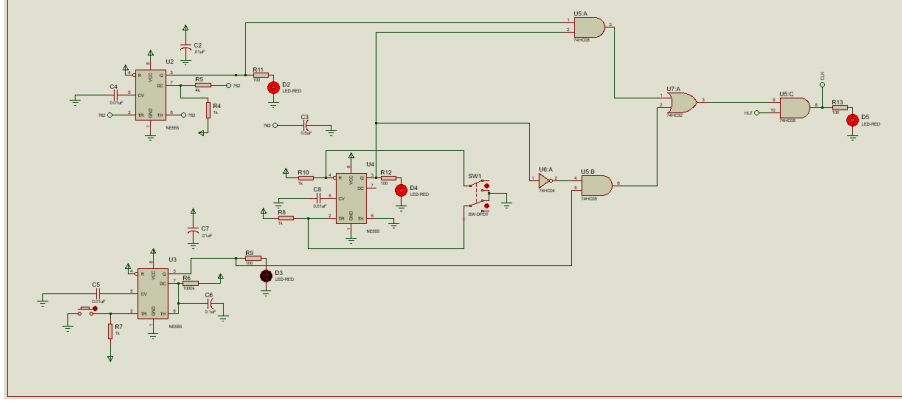


Figure 2

The clock module is the timing source for the entire computer, generating the master clock signal that synchronizes every register transfer in the system. A single free-running oscillator is not sufficient on its own: for development and debugging purposes, the system also requires the ability to advance the clock one pulse at a time under manual control, to switch cleanly between automatic and manual operation without introducing spurious pulses, and to halt entirely when the control logic asserts a HLT signal. The clock module was therefore designed around three separate uses of the 555 timer IC, one for each of these requirements, combined through discrete logic gating into a single clock output.

Astable Configuration: Free-Running Clock

The automatic clock source is a 555 timer[3] configured in astable mode, producing a continuous square wave whose frequency is set by two resistors and a capacitor. With the timing resistor connected from V_{CC} to the discharge pin (R_1) and a second resistor forming the charge and discharge path to the threshold and trigger pins (R_2), the output period is given by

$$T = 0.693 \times (R_1 + 2R_2) \times C$$

Using $R_1 = 1 \text{ k}\Omega$, a potentiometer for R_2 in the range of $1 \text{ k}\Omega$ to $100 \text{ k}\Omega$, and a timing capacitor in the range of tens of nanofarads to a few microfarads (selected depending on whether a fast run speed or a slow, human-observable speed is desired), this circuit produces a variable-frequency square wave suitable for automatic program execution. A $0.01 \mu\text{F}$ decoupling capacitor on the control-voltage pin and a $0.1 \mu\text{F}$ bypass capacitor across the supply pins were included, following the manufacturer's datasheet recommendations, to suppress switching noise on the comparator reference and reduce supply-line transients.

Monostable Configuration: Debounced Manual Pulse

Manual single-stepping is provided by a second 555 timer configured in monostable mode. In this configuration, a falling edge on the trigger input produces a single output pulse of fixed duration, given by

$$T \approx 1.1 \times R \times C$$

With $R = 1\text{ M}\Omega$ and $C = 0.1\text{ }\mu\text{F}$, this yields a pulse width of approximately 110 ms, which is well within the timing window required to reliably clock the rest of the system from a single button press. The key advantage of this configuration over a raw mechanical switch is its inherent debouncing capability. Because the monostable output duration is determined by the RC time constant rather than by how long the button is physically held, contact bounce occurring immediately after the button press is prevented from generating multiple clock pulses and therefore does not reach the rest of the circuit.

Bistable Configuration: Debounced Mode-Select Switch

Selecting between automatic and manual operation requires a clean, debounced toggle signal. Rather than introducing a separate debounce circuit for this switch, a third 555 timer was configured in bistable mode, using only its internal SR latch. In this mode, the threshold pin is tied to ground, permanently disabling the upper comparator, while the trigger and reset pins are driven directly by the two throws of a single-pole double-throw (SPDT) switch, each pulled up through a $1\text{ k}\Omega$ resistor. Because the SR latch responds only to the first edge on either input and then holds its state, mechanical bounce on the switch contacts has no effect on the output. The latch therefore changes state exactly once per switch actuation, regardless of contact bounce.

Combining the Clock Sources

The outputs of the three 555 circuits are combined using discrete NOT, AND, and OR gates (74LS04, 74LS08, and 74LS32) to form the final clock signal, implementing the logic function

$$\text{CLK} = [(\text{Select} \cdot \text{Auto}) + (\overline{\text{Select}} \cdot \text{Manual})] \cdot \overline{\text{HLT}}$$

The bistable output (Select) and its complement gate one AND term each to the automatic and manual clock sources, respectively, ensuring that exactly one source drives the output at any time. The resulting signal is then ANDed with the inverted HLT line, so that when the control unit asserts HLT, the final AND gate is forced low regardless of the state of either clock source, halting the machine until the system is reset.

This design allows the operator to run the computer freely at an adjustable speed, single-step through a program one instruction cycle at a time for debugging, and switch between the two modes at any point without risk of a spurious or double clock pulse being generated during the transition.

3 A Register and B Register Design

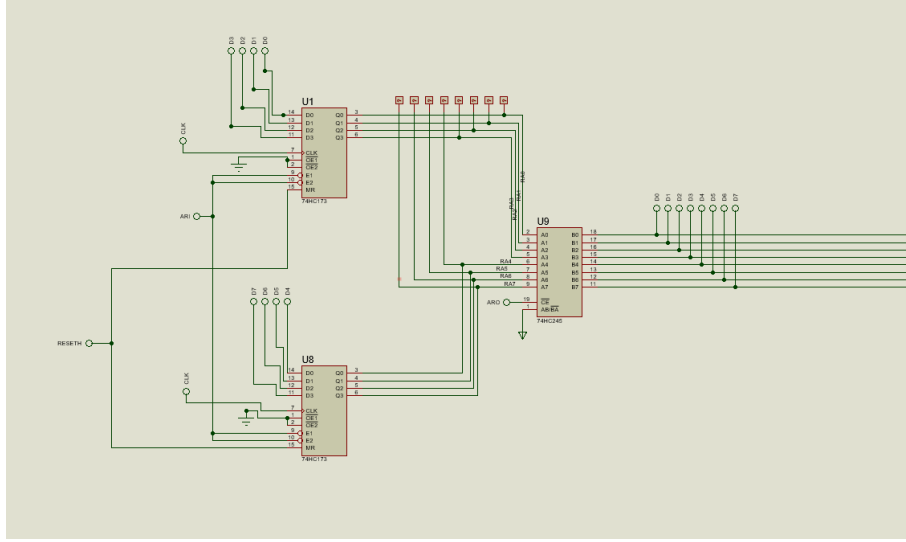


Figure 3

The A register (accumulator) and B register together form the operand-holding stage of the arithmetic logic unit. Both are 8-bit storage elements connected to the shared system bus, and each must satisfy three requirements common to every bus-connected register in the design: the ability to capture a new value from the bus on a specific clock edge only when explicitly instructed to do so, the ability to hold that value indefinitely otherwise, and, where required, the ability to drive that stored value back onto the bus without interfering with any other module sharing the same lines. The A and B registers were implemented identically at the circuit level, using two 4-bit registers and one tri-state bus buffer per register, differing only in how their control signals are decoded from the microcode and how their outputs are used elsewhere in the datapath.

Storage: 74HC173 Load-Selection Logic

Each register is built from two 74HC173[4] 4-bit D-type registers, providing a full 8 bits of storage. Internally, each bit of the 74HC173 is preceded by a small multiplexing circuit that selects, on every active clock edge, between the register’s current output and the new value present at its data input. This selection is controlled by the register’s active-low data-enable (load) pins:

$$D_{\text{next}} = \begin{cases} Q_{\text{current}}, & \text{if LOAD is inactive,} \\ D_{\text{input}}, & \text{if LOAD is active.} \end{cases}$$

When the load signal is inactive, the register retains its existing value on each clock pulse, so its contents remain unchanged even though the clock continues to run. When the load signal is active, the next clock edge captures the value currently present on the bus. This mechanism allows a single shared master clock to drive every register in the system continuously, while the microcode’s IN control signals determine, on a cycle-by-cycle basis, which specific registers actually update.

Bus Interfacing: 74HC245 Tri-State Output

The output side of each register is handled by a 74HC245 octal bus transceiver, configured for one-way operation by tying its direction pin permanently to the supply rail. This chip's outputs are tri-state: in addition to driving a logic 0 or 1, they can be placed in a high-impedance state in which the pin neither sources nor sinks current and is effectively disconnected from the line it is wired to. This property is essential for correct bus operation, since more than one register's output is permanently wired to the same eight bus lines; without tri-state capability, every register would attempt to actively drive the bus simultaneously, resulting in electrical contention whenever two or more disagreed on a bit's value. The 74HC245's[5] enable pin, driven by the register's OUT control signal, is asserted only for the single register that has been selected to place its contents on the bus during that clock cycle; every other bus-connected module's tri-state buffer remains disabled at that instant, leaving the bus lines free.

Load and Output Control Signals

Both registers receive their load signal (active-low, tying both data-enable pins of the 74HC173[4] pair together) from a dedicated control-word bit generated by the microcode ROM: ARI (A register in) and BRI (B register in). The A register's tri-state output is enabled by ARO (A register out), used both when transferring a value from A back onto the bus and, in the case of the OUT instruction, when routing the accumulator's contents to the output register. The B register's tri-state output, gated by an equivalent enable signal, is included for architectural symmetry with the other bus-connected modules, although in the instruction set implemented in this project the B register's contents are consumed directly by the ALU and are not required to be placed back on the bus by any instruction.

Interaction with the ALU

Both registers' two-state (non-tri-state) internal outputs are wired directly and permanently to the adder/subtractor inputs, independent of whether their tri-state bus outputs are currently enabled. This means the ALU continuously computes the sum or difference of whatever values are presently stored in A and B, regardless of bus activity; the ALU's own output is only actually used, however, at the specific microcode step where the sum-out control signal is asserted, gating the result onto the bus for capture back into the A register.

This design allows the A and B registers to independently and safely share the single 8-bit system bus with every other register in the machine, using the same load-gating and tri-state-enable pattern that is reused, with minor variation, across the instruction register, memory address register, program counter, and output register described in later sections.

4 Arithmetic Logic Unit (ALU) Design

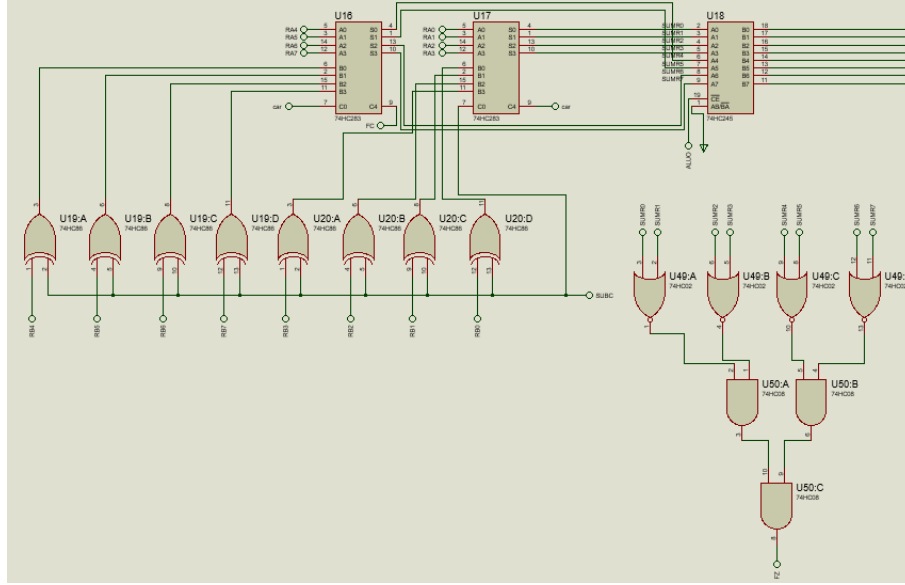


Figure 4

The arithmetic logic unit is responsible for computing the sum or difference of the values held in the A and B registers, on demand, and for making that result available both to the rest of the datapath (for capture back into the A register) and to the flags register, which records whether the result overflowed 8 bits or evaluated to zero. Unlike the register modules described previously, the ALU is a purely combinational circuit: it has no clock input of its own and continuously recomputes its output as soon as either operand changes, with the timing of when that result is actually used determined entirely by the microcode.

8-Bit Addition: Cascaded 4-Bit Adders

The ALU is constructed from two 74HC283 4-bit full adders cascaded to form an 8-bit adder. The lower 74HC283 processes bits 3:0, while the upper 74HC283[6] processes bits 7:4. The carry-out of the lower adder, C_4 , is connected to the carry-in of the upper adder. Thus, the two stages can be expressed as

$$(C_4, S_{3:0}) = A_{3:0} + B_{3:0} + C_{in}$$

$$(C_8, S_{7:4}) = A_{7:4} + B_{7:4} + C_4$$

where C_4 is the carry propagated from the lower nibble to the upper nibble, and C_8 is the final carry-out from bit 7. Consequently, the complete 8-bit addition is

$$(C_8, S_{7:0}) = A_{7:0} + B_{7:0} + C_{in}.$$

The final carry-out, C_8 , is used as the Carry flag of the ALU and indicates that an unsigned result exceeds the 8-bit range.

Subtraction via Two's Complement

Rather than duplicating the adder hardware for subtraction, the ALU implements subtraction by conditionally inverting the B operand before it reaches the adder, exploiting the two's-complement identity

$$A - B = A + (\overline{B} + 1).$$

The bitwise complement of B is produced by a bank of XOR gates, one for each bit. One input of each XOR gate is connected to the corresponding bit of the B operand, while the other input is connected to a common subtraction control signal, SUBC:

$$B'_i = B_i \oplus \text{SUBC}.$$

When SUBC is low, each XOR gate passes its corresponding B bit unchanged, so the adder performs an ordinary addition:

$$A + B + C_{\text{in}}.$$

When SUBC is high, every B bit is inverted, producing the one's complement $\overline{B}$. The same SUBC signal is simultaneously applied to the carry-in of the first (lower-nibble) 74HC283, providing the required $+1$ term. Therefore, the adder performs

$$A + \overline{B} + 1 = A - B.$$

This arrangement allows the same 8-bit adder chain to perform both addition and subtraction. A single control signal, SUBC, simultaneously selects whether the B operand is passed unchanged or complemented and whether the initial carry-in is asserted, eliminating the need for a separate subtraction circuit.

Tri-State Output to the Bus

The 8-bit result is connected to the system bus through a 74HC245[5] tri-state buffer, in the same configuration used for the A and B registers: direction fixed for one-way transfer, and the enable pin driven by a dedicated control signal, ALUO (ALU out). Because the adder chain is combinational, the value present at the buffer's input is always the current sum or difference of A and B, regardless of whether ALUO is asserted; the enable signal only determines whether that value is permitted to drive the bus at a given clock cycle, preventing contention with any other bus-connected module.

Flag Generation

Two status conditions are derived from the ALU's output and made available to the flags register described in a later section. The carry flag is taken directly from the final carry-out of the high-nibble 74HC283[6] (pin C4 of the second adder), representing an unsigned overflow of the 8-bit result. The zero flag is generated by a separate zero-detection circuit: the eight sum bits are paired and combined through four 2-input NOR gates, and the four NOR outputs are combined through a tree of three 2-input AND gates, producing a single signal that is high only when every one of the eight sum bits is zero. Both signals are latched into the flags register on the same clock edge that captures the ALU result into the A register, ensuring that the flags always correctly reflect the outcome of the most recently executed arithmetic instruction.

5 Program Counter Design

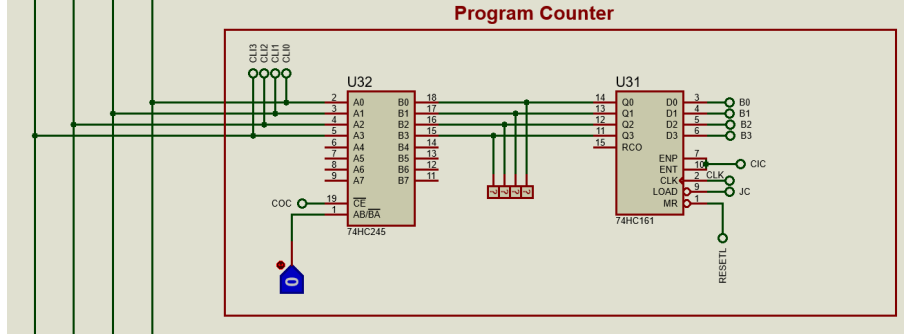


Figure 5

The program counter maintains the address of the next instruction to be fetched from memory, incrementing automatically as the machine progresses through a program while also permitting its value to be overwritten directly, an operation required to implement the jump instructions. As with the other bus-connected registers described previously, the program counter must be able to place its contents on the shared system bus on demand without contending with any other module, and its contents must update synchronously with the rest of the machine on the shared master clock.

Counting: 74HC161 Synchronous 4-Bit Counter

The program counter is implemented using a single 74HC161[7] synchronous 4-bit binary counter, rather than a discrete arrangement of JK flip-flops. This choice was made for two reasons. First, the 74HC161 integrates the count-enable, parallel-load, and asynchronous clear functions required by the design into a single chip, in place of the additional gating logic that would otherwise be needed around a bank of discrete flip-flops. Second, and more importantly, the 74HC161 updates its output on the rising edge of the clock, consistent with every other synchronous register in the system; a counter built from master-slave JK flip-flops in the conventional ripple configuration would instead change state on the falling edge, introducing a timing mismatch that could cause the program counter to update out of phase with the rest of the machine.

Only four bits of address are required, since the system’s RAM has sixteen locations; the counter’s ripple-carry output (RCO) is therefore unused, as cascading to a wider counter is unnecessary at this memory size.

Counting and Jump Behavior

The counter’s count-enable input, driven by the microcode control signal CIC (count-increment control), determines whether the program counter advances on a given clock edge. This signal is asserted during the first step of every instruction’s fetch cycle, ensuring that the program counter increments exactly once per instruction, regardless of the number of additional clock cycles required for instruction execution:

$$PC_{\text{next}} = \begin{cases} PC + 1, & \text{if CIC is active,} \\ PC, & \text{otherwise.} \end{cases}$$

To support jump instructions, the counter's parallel-load input is driven by the control signal JC (jump control). When asserted, the next clock edge loads the counter's four data inputs, taken from the low nibble of the instruction register, instead of incrementing the counter:

$$PC_{\text{next}} = \begin{cases} \text{Operand}_{3:0}, & \text{if JC is active,} \\ PC + 1, & \text{if CIC is active and JC is inactive,} \\ PC, & \text{otherwise.} \end{cases}$$

On the 74HC161, the parallel-load input takes priority over the count-enable input whenever both are asserted on the same clock edge. Although the microcode in this design does not intentionally assert both signals simultaneously, this built-in priority ensures that a jump instruction takes effect correctly rather than being overridden by a counter increment.

Bus Interfacing: 74HC245 Tri-State Output

Because the 74HC161's own outputs are not tri-state, the counter's four bits are connected to the bus through a 74HC245 buffer, configured identically to the tri-state stages used in the A and B registers and the ALU. The buffer's enable pin is driven by the control signal COC (counter out control), asserted during the address-transfer step of every fetch cycle to place the program counter's value onto the bus for capture into the memory address register, and also asserted, together with the jump control signal on the opposite side of the transfer, whenever the current instruction's own jump target needs to reach the program counter's data inputs via the bus.

Reset

The counter's asynchronous clear input is connected to the shared system reset network described in an earlier section, allowing the program counter to be returned to address zero at the start of every program run independently of the clock.

6 RAM Module Design

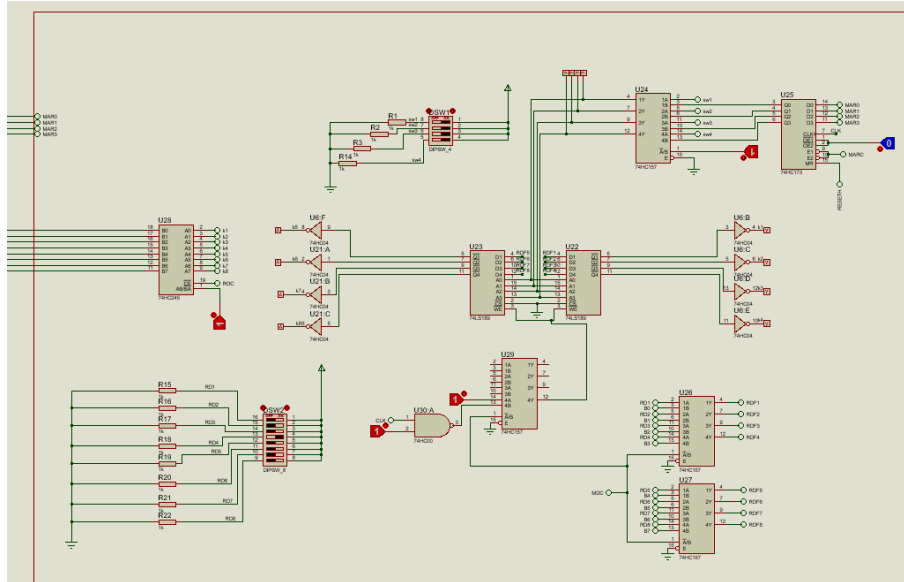


Figure 6

The RAM module provides the sixteen bytes of program and data storage from which the machine fetches instructions and reads or writes operands during execution. Beyond simple storage, the design must satisfy two additional requirements specific to this project: the memory's address and data inputs must be switchable between two distinct sources, either DIP switches under direct operator control, for manually loading a program before a run, or the live system bus, for normal instruction execution, and the write operation itself must be synchronized to the system clock rather than triggered immediately whenever a write is requested.

6.1 Storage: Cascaded 74LS189 Static RAM

The sixteen bytes of storage are implemented using two 74LS189[8] 64-bit static RAM chips, each organized as sixteen 4-bit words. The two chips share the same four address lines, so that a given address always selects the same row in both, while each chip supplies one nibble of the resulting byte, together forming a full 16-word by 8-bit memory.

Both chips' active-low chip-select inputs are tied permanently to ground, so that the memory is always selected and available to respond to read and write operations. A characteristic of the 74LS189 is that its output data is the logical complement of the stored value; this is corrected by routing each of the eight output lines through a 74HC04 inverter before the data reaches the rest of the module, so that the value read out matches the value originally written.

6.2 Data Selection and Synchronized Write Control

The eight data input lines are handled identically to the address lines, using two further 74HC157 multiplexers (U26, U27) selecting between a bank of eight DIP switches (DSW2), for manually entering a byte to be written, and the live system bus, for data supplied by the running program.

The same mode-select line used for address selection governs this choice as well, so that address and data switch sources together and remain consistent with one another.

Writing to RAM: MDC and Synchronized Write Control

A byte is written into the addressed RAM location when the microcode ROM asserts a dedicated control signal, referred to in this design as **MDC** (Memory Data Control), the write-side counterpart to MARC, which loads a new address into the memory address register. Where MARC captures a value from the bus into the MAR, MDC governs whether the byte currently on the bus is committed into the RAM location the MAR is pointing to.

MDC originates as one bit of the control word produced by the microcode ROM (labeled MDCC prior to buffering) and is carried through the same buffer and inversion stage used for every other control signal in the design. Because the 74LS189's write-enable input is level-sensitive rather than edge-triggered, MDC cannot be connected directly to it: doing so would cause the memory to write continuously for as long as MDC remained asserted, independent of the clock. Instead, MDC is combined with the system clock through a NAND gate (U30:A):

$$\overline{\text{WE}} = \overline{\text{MDC}} \cdot \overline{\text{CLK}}$$

This confines the resulting write pulse to the portion of the clock cycle during which both signals are simultaneously high, so that writes to memory occur synchronously with the rest of the machine's register transfers, following the same clock-edge convention used everywhere else in the design. The output of this NAND gate is itself routed through a further 74HC157[9] multiplexer (U29), which selects between this program-generated write pulse and a manual push-button used when the operator writes a byte into memory by hand while entering a program via the DIP switches. The same mode-select line that governs the address and data multiplexers also governs this selection, so that the entire memory subsystem, address, data, and write control together, switches consistently between manual programming mode and normal program execution.

Bus Interfacing: 74HC245 Tri-State Output

The corrected (re-inverted) output data is connected to the system bus through a 74HC245 tri-state buffer (U28), in the same configuration used throughout the rest of the datapath, with its enable pin driven by the control signal ROC (RAM out control), asserted whenever the currently addressed byte needs to be placed on the bus for use elsewhere in the machine.

The Complete Write Path

Bringing the above together, writing a value into memory during program execution (for example, during execution of the STA instruction) proceeds as follows: the value to be stored, typically the contents of the A register, is placed on the system bus; the address to write to is already held in the memory address register, set during an earlier microcode step; the data-select multiplexers route this bus value to the RAM chips' data inputs, since the mode switch is set to program-execution mode; and the microcode asserts MDC during the appropriate step of the instruction's execution, which, gated by the clock through the NAND stage and routed through the write-select multiplexer, produces a single synchronized write pulse into the addressed RAM location.

6.3 Memory Address Register (MAR)

The memory address register is responsible for holding the four-bit address that both memory chips currently respond to, supplying it to the address-select multiplexer whenever the module is switched into program-execution mode. It is implemented using a single 74HC173[4] 4-bit register

(U25), following the same load-selection pattern used by the A and B registers, the instruction register, and every other bus-connected register in the design.

The register's output-control pins are tied permanently low, so that its four outputs continuously drive their destination, rather than being tri-stated; unlike the A or B registers, the MAR's outputs never need to be disconnected from the address-select multiplexer, since that multiplexer's own select line, not the MAR's own tri-state control, is what determines whether the MAR's value actually reaches the RAM. The MAR's data-enable (load) pins are driven by the control signal MARC (memory address register control), asserted by the microcode whenever a new address, typically the low nibble of the instruction register or the current value of the program counter, needs to be latched into the MAR from the system bus. As with every other synchronous register in the design, the MAR's clock input is tied to the shared master clock, so that the new address is captured on the same clock edge used throughout the rest of the machine, and its asynchronous clear input is connected to the shared system reset network, allowing the MAR to be returned to a known state independently of the clock at the start of a program run.

Once loaded, the MAR's four output bits feed directly into the "program execution" side of the address-select multiplexer (U24) described previously, so that whenever the mode switch is set away from manual DIP-switch addressing, it is this stored value, rather than the address currently on the bus, that continues to select the active RAM row for as long as the MAR is not reloaded, regardless of what subsequently changes elsewhere on the bus.

7 Instruction Register Design

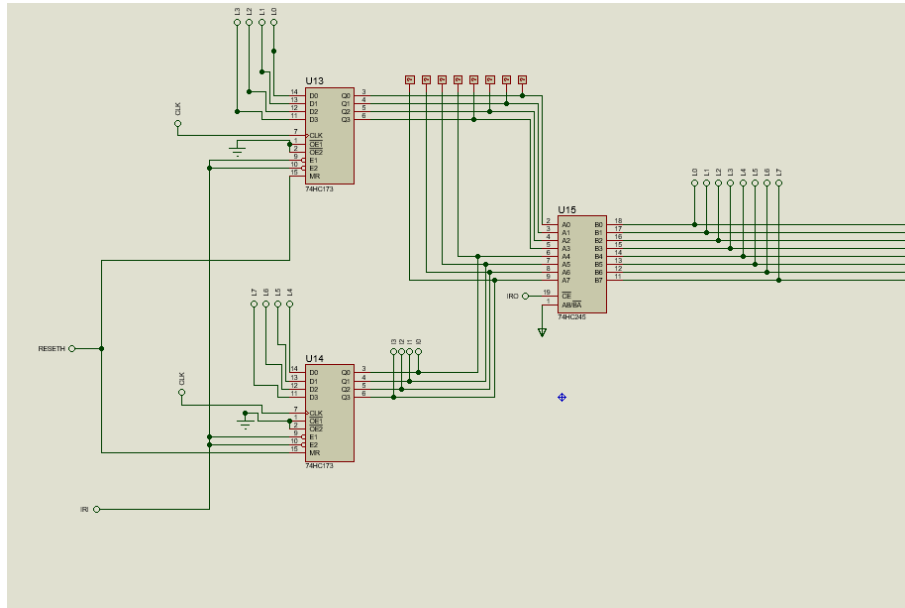


Figure 7

The instruction register holds the byte most recently fetched from memory for the duration of that instruction's execution, making it available to two separate parts of the machine simultaneously: the four most significant bits, the opcode, must reach the control unit's microcode ROM,

where they are used, together with the step counter and flag state, to determine the control word for every subsequent step of the instruction's execution, while the four least significant bits, the operand or address field, must be available on the system bus whenever an instruction needs to reference a memory location, an immediate value, or a jump target. Unlike the A and B registers, the instruction register therefore has an asymmetric relationship with the rest of the datapath: only half of its contents are ever placed on the bus, while the other half is routed permanently to a separate destination.

Storage: 74HC173 Load-Selection Logic

The instruction register is implemented using two 74HC173 4-bit registers, following the same load-selection pattern used throughout the rest of the design. As with the A and B registers, each bit's internal multiplexer selects between the register's own current output and its data input on every clock edge, controlled by the active-low load pins:

$$D_{\text{next}} = \begin{cases} Q_{\text{current}} & \text{if IRI is inactive} \\ D_{\text{input}} & \text{if IRI is active} \end{cases}$$

Both 74HC173 chips' load pins are tied together and driven by the single control signal **IRI** (instruction register in), asserted by the microcode during the second step of every instruction's fetch cycle, so that the next clock edge captures the byte currently being read out of RAM at the address held in the memory address register. The two chips' clock inputs are tied to the shared system clock, and their asynchronous clear inputs are connected to the shared system reset network, consistent with every other register in the design.

The Opcode Field: Direct Connection to the Control Unit

The upper four bits, produced by one of the two 74HC173 chips, are not routed through any tri-state buffer or multiplexer. Instead, they are wired permanently and directly to four of the address lines of the microcode ROM described in the control unit section of this report. This is a deliberate departure from the pattern used by the bus-connected registers described elsewhere: because the opcode's only role is to select which routine the control unit is currently executing, there is no circumstance in this design under which the opcode needs to appear on the system bus, and no tri-state gating is required to prevent contention, since this connection is never shared with any other driver.

The Operand Field: Tri-State Output to the Bus

The lower four bits, produced by the second 74HC173, are connected to the system bus through a 74HC245 tri-state buffer (U15), in the same configuration used by the A register, B register, ALU, and program counter. This chip's enable pin is driven by the control signal **IRO** (instruction register out), asserted whenever the currently held instruction's operand field needs to be placed on the bus, for example, to supply a memory address to the memory address register during the execution of LDA, STA, ADD, or SUB, to supply an immediate value directly to the A register during LDI, or to supply a jump target to the program counter during JMP, JC, or JZ. Only the four least significant bus lines are ever driven by this buffer; the remaining four lines of the 74HC245 are left unused, since the instruction register's operand field, and every value it is ever required to supply, fits within four bits given the system's sixteen-byte address space.

8 Output Module and Display Design

The output module is responsible for capturing the final result of a program's execution from the system bus and presenting it to the operator in human-readable decimal form. Unlike every other bus-connected register in the design, the output register never needs to place its own contents back onto the bus, since it exists purely as a data sink at the end of the datapath; this allows its internal storage to be implemented more simply than the A, B, instruction, and address registers described previously. Beyond storage, the module must also translate the stored 8-bit binary value into a human-readable multi-digit decimal display, a task that, if implemented with discrete decoding logic for three or four digits simultaneously, would require a substantial number of additional gates. The design instead uses a single EEPROM as a combinational lookup table, together with a small amount of dedicated multiplexing hardware, to drive a set of common-cathode seven-segment displays from that one chip.

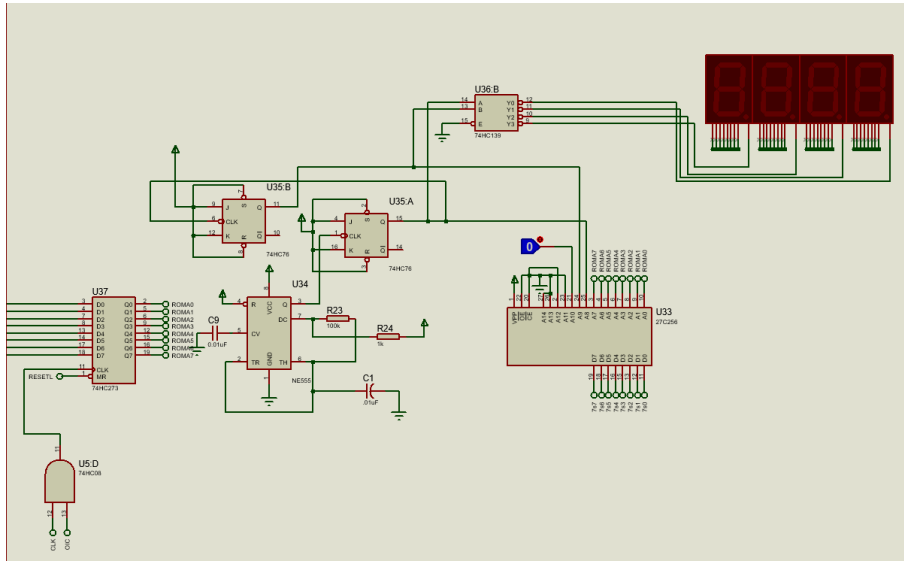


Figure 8

8.1 Output Register

The output register is implemented using a single 74HC273 octal D-type flip-flop (U37), rather than the pair of 74HC173 registers used elsewhere in the design. Because the output register never drives the bus, the tri-state bus-interfacing stage (74HC245) used by every other register is unnecessary here, and a single 8-bit flip-flop chip is sufficient.

The 74HC273, however, has only a single clock input and no separate load-enable pin of the kind provided by the 74HC173. Left connected directly to the system clock, the register would therefore capture whatever value happened to be on the bus on every single clock pulse, rather than only when the OUT instruction is executed. To prevent this, the register's clock input is instead driven by the output of an AND gate combining the system clock with the microcode control signal **OIC** (output-in control):

$$\text{CLK}_{\text{out reg}} = \text{CLK} \cdot \text{OIC}$$

Only when OIC is asserted does a rising system-clock edge reach the register's clock pin at all; on every other cycle, the gated clock remains low and the register's stored value is left undisturbed, regardless of what is currently on the bus. The register's asynchronous master-reset input is tied to the shared system reset network, and its data inputs are connected directly to the eight bus lines.

8.2 Decimal Decoding: EEPROM Lookup Table

Rather than building discrete segment-decoding logic, the eight bits held in the output register are used directly as part of the address input to a 27C256[10] EEPROM (U33), which is pre-programmed so that its eight data outputs, at any given address, hold exactly the seven-segment pattern required to display the corresponding digit. This is possible because any combinational function, including the mapping from an 8-bit value to the pattern of segments needed to display it, can be implemented as a lookup table: the input becomes the address, and the desired output is simply written into memory at that address ahead of time.

The eight output-register bits form the lowest eight address lines of the EEPROM (A0 through A7), so that reading the chip at that address returns the segment pattern for the currently stored value. The remaining address lines used by this module select which of several stored tables is being read, allowing the same chip to serve more than one purpose, described in the following three subsections.

8.3 Choice of EEPROM: 27C256

The 27C256 was selected as the EEPROM for both the output display's lookup table and the control unit's microcode ROM (described in a later section) for two practical reasons specific to this project, rather than for any electrical property that distinguishes it from the other EEPROM parts considered.

First, the version of Proteus used for this simulation only provides a working SPICE model for the 27C256 among the EEPROM parts available in its component library; other common choices, such as the 28C16 used in the original hardware design this project follows, are present in the schematic capture library but do not simulate correctly, since Proteus lacks the corresponding behavioral model needed to reproduce their memory contents during a transient simulation. Since the entire computer in this project exists only as a Proteus simulation rather than a physical breadboard build, the ability to actually simulate the chip's stored contents took priority over matching the specific part used in the reference hardware design.

Second, and more significantly for the development workflow, Proteus allows a 27C256 component's memory contents to be initialized directly from a binary file specified in the component's properties, without requiring any physical programming step. This made it possible to generate the required lookup table contents programmatically, for both the seven-segment decoding table and the microcode control word table, and load the resulting binary file directly into the simulated chip. In a physical build, populating an EEPROM with this kind of large, systematically generated lookup table would typically require a dedicated hardware programmer, such as an Arduino-based EEPROM programmer built specifically for this purpose. Because this project is implemented entirely in simulation, that hardware programming step is unnecessary: the same binary file that would otherwise be burned onto a physical chip can instead be attached directly to the simulated component, considerably simplifying the process of generating, testing, and iterating on both lookup tables during development.

8.4 Multiplexed Multi-Digit Display

Displaying a value larger than nine digits requires more than one seven-segment display, but wiring a separate EEPROM decoder to each digit would multiply the hardware required. Instead, all digits share the same eight segment-driving lines from the EEPROM, and only one digit at a time is enabled to actually respond to those lines, cycled fast enough that persistence of vision makes the digits appear to be lit simultaneously.

This multiplexing is driven by a dedicated 555 timer (U34) configured in astable mode, deliberately independent of the main system clock, since the multiplexing rate is a display concern unrelated to the speed at which the computer itself is executing instructions. The 555's output clocks a 2-bit counter built from a 74HC76 dual JK flip-flop (U35), used in toggle mode, whose two output bits serve two purposes simultaneously: they are connected to two further EEPROM address lines (A8 and A9), selecting which digit's value the EEPROM should currently be decoding, and they are also connected to the inputs of a 74HC139 2-to-4 line decoder (U36), whose four outputs each enable one digit's common cathode.

Because the same two counter bits drive both the EEPROM's digit-select address lines and the decoder's digit-enable inputs, the segment pattern being output by the EEPROM and the digit currently allowed to display it are always synchronized: at any instant, exactly one digit is enabled, and the EEPROM is simultaneously outputting that same digit's correct pattern, so that no other digit briefly displays an incorrect value during the transition.

8.5 Signed and Unsigned Display Modes

A further EEPROM address line (A10) selects between two independently programmed regions of the same chip, one containing the standard unsigned decimal decoding used throughout the rest of this report, and a second containing a two's-complement signed decoding, in which a value in the upper half of the 8-bit range is displayed as its negative equivalent, with a dedicated segment pattern reserved for the minus sign on the leftmost digit position. This selection line is exposed as a switch, allowing the operator to choose the interpretation appropriate to the program currently running.

9 Control Unit Design

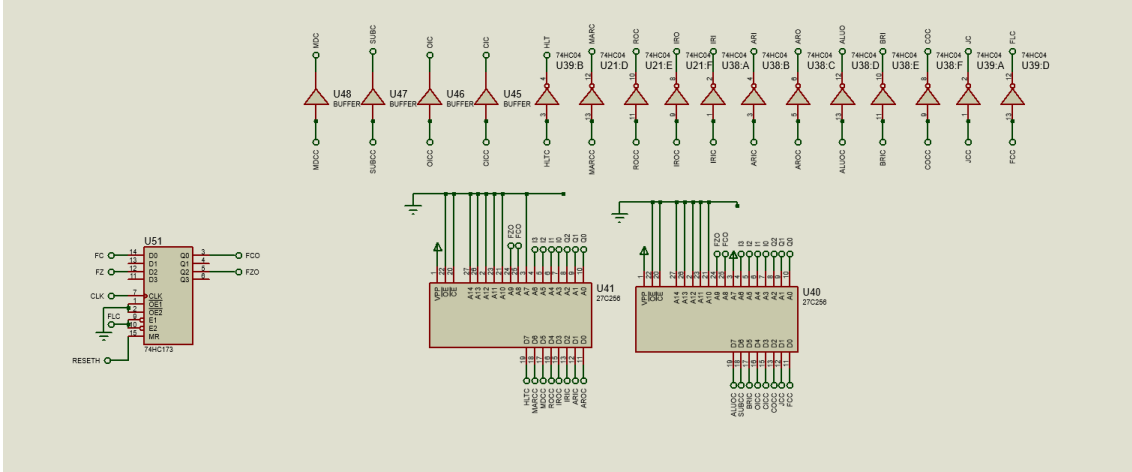


Figure 9

The control unit is responsible for translating the currently fetched instruction, together with the machine’s progress through that instruction’s execution and the current state of the ALU’s status flags, into the specific set of control signals that must be active on any given clock cycle. Rather than implementing this translation as hardwired combinational logic, as in Malvino’s original control matrix design described in the introduction, this project follows a microprogrammed approach: the entire mapping from instruction, step, and flag state to control word is stored as a lookup table in ROM, and the control unit’s task reduces to generating the correct address into that table on every clock cycle.

9.1 Step Counter

Progress through an instruction’s execution is tracked by a 74HC161[7] synchronous 4-bit counter (U43), operating independently of the program counter described previously and serving a different purpose: rather than addressing memory, its output selects which step, T0 through T4, of the current instruction’s microcode is currently active. The counter’s three least significant bits are used; its count-enable input is tied permanently active, so that it advances on every clock cycle, and its clear input is connected to a dedicated reset network that returns the counter to zero at the start of every instruction’s fetch cycle as well as at system reset. Because every instruction in this design is executed in exactly five steps regardless of how many of those steps it actually uses, the same counter and the same addressing scheme serve every instruction in the set without exception.

9.2 Microcode ROM Addressing

The control word for a given clock cycle is generated by two 27C256 EEPROMs (U40, U41), addressed identically and in parallel. Three sources are combined to form the ROM address:

$$\text{Address} = \text{Step}_{2:0} \parallel \text{Opcode}_{3:0} \parallel \text{Carry} \parallel \text{Zero}$$

The three least significant address lines (A0-A2) are driven by the step counter; the next four (A3-A6) are driven directly by the instruction register’s opcode nibble, described in the previous

section; and the two highest lines used (A8, A9) are driven by the flags register's latched carry and zero outputs (FCO and FZO), described in the following subsection. This addressing scheme is what allows the conditional-jump instructions to work without any additional decision logic outside the ROM itself: for every combination of step and opcode where the flag state is irrelevant, the same control word is programmed at both possible values of the flag bits, while for the JC and JZ instructions specifically, the programmed control word differs depending on whether the relevant flag is set, so that the jump either does or does not occur purely as a consequence of which ROM address is presented.

9.3 Byte-Select Addressing and Identical ROM Programming

Since the resulting control word is sixteen bits wide but each EEPROM provides only eight data outputs, one further address line (A7) is used to select which half of the word a given chip returns at a given address. Both U40 and U41 are programmed with an identical binary image; the same instruction, step, and flag combination therefore produces the same eight-bit value on both chips whenever A7 is 0, and a different, complementary eight-bit value whenever A7 is 1. This is achieved in practice by hardwiring A7 to ground on one chip and to the supply rail on the other, so that each physical chip is permanently restricted to only ever reading one half of the shared sixteen-bit word at any given (step, opcode, flag) address, despite both chips containing the same programmed contents. This approach was chosen specifically because it allows both ROMs to be programmed from a single generated binary file, removing any possibility of the two chips being programmed inconsistently with one another and eliminating the need to track which of two separate files belongs in which socket.

Both chips' output-enable and chip-enable pins are tied permanently active, and their write-enable pins are tied permanently inactive, since the ROM contents are fixed for the duration of a simulation run and are never written to during normal operation.

9.4 Control Signal Buffering

The sixteen raw outputs produced by the two ROMs are not connected directly to the registers and multiplexers they control. Instead, each output is passed through a dedicated 74HC04 inverter or buffer stage before reaching its destination, as shown in the accompanying schematic. This buffering serves two purposes: first, it provides the drive strength needed to fan the ROM's output out to whichever module actually consumes that signal, without loading the ROM's own output pins directly; second, where a destination register expects an active-low control input, such as the load-enable pins on the various 74HC173 registers throughout the design, the corresponding buffer stage is configured as an inverter rather than a straight buffer, so that the ROM itself can be programmed uniformly with active-high logic while still correctly driving the mixture of active-high and active-low control inputs present elsewhere in the machine.

9.5 Control Signal Reference Table

Every signal listed in the table is asserted during specific (step, opcode, flag-state) combinations, as determined by the contents of the microcode ROM. The fetch-cycle signals (MARC and COC at step 0; ROC, IRI, and CIC at step 1) are common to every instruction, while the remaining signals are asserted only during the execution steps of the specific instructions that require them. As described previously, FCC is unique among these sixteen bits in that it does not drive a register or multiplexer directly, but is instead inverted through a dedicated buffer stage (U39:D) to produce FLC, asserted only during the final step of the ADD and SUB instructions, so that the flags register captures a new carry and zero result only at the precise instant a new arithmetic result becomes valid.

Table 2: Control Word Signal Reference

Signal	Full Name	Destination	Function
HLT	Halt	Clock module	Stops the system clock, halting execution
MARC	Memory Address Register Control	MAR (74HC173)	Loads a new address from the bus into the MAR
MDC	Memory Data Control	RAM write path	Enables a synchronized write of the bus value into the addressed RAM location
ROC	RAM Out Control	RAM tri-state buffer	Places the addressed RAM byte onto the bus
IRI	Instruction Register In	Instruction register (74HC173)	Loads the fetched byte from the bus into the instruction register
IRO	Instruction Register Out	Instruction register tri-state buffer	Places the operand nibble onto the bus
ARI	A Register In	A register (74HC173)	Loads a new value from the bus into the A register
ARO	A Register Out	A register tri-state buffer	Places the A register's contents onto the bus
ALUO	ALU Out	ALU tri-state buffer	Places the adder/subtractor result onto the bus
SUBC	Subtract Control	ALU XOR bank	Selects subtraction instead of addition
BRI	B Register In	B register (74HC173)	Loads a new value from the bus into the B register
OIC	Output-In Control	Output register clock gate	Gates the system clock into the output register (74HC273)
COC	Counter Out Control	Program counter tri-state buffer	Places the program counter's value onto the bus
CIC	Count Increment Control	Program counter (74HC161)	Enables the program counter to increment
JC	Jump Control	Program counter (74HC161)	Loads a new address from the bus into the program counter
FCC → FLC	Flags Control (inverted to Flags Load Control)	Flags register (74HC173, U51)	Loads the ALU's carry and zero outputs into the flags register

10 Reset Module Design

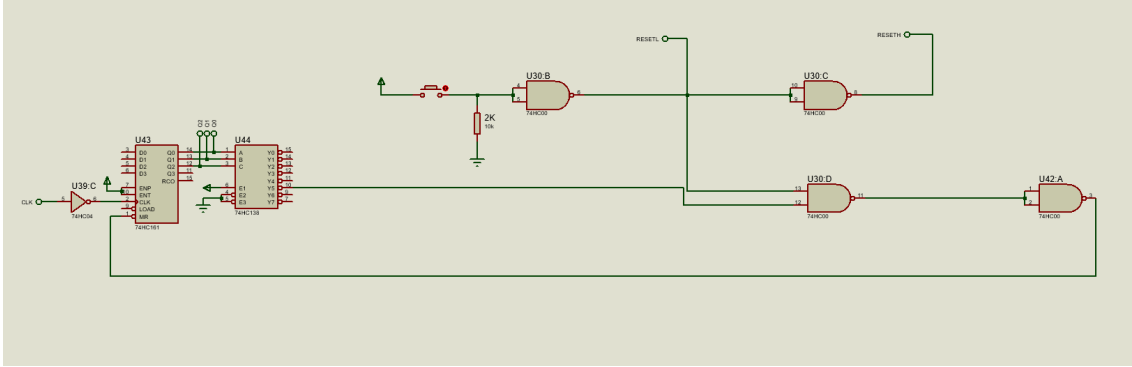


Figure 10

The reset module serves two related but distinct purposes within the machine. First, it provides a single, debounced, operator-triggered reset that returns every register and counter in the system, the program counter, A and B registers, instruction register, memory address register, flags register, and step counter, to a known state at the start of a program run, without requiring the operator to reset each module individually. Second, and independently of the manual reset button, it must automatically reset the step counter at the end of every instruction cycle, so that execution returns to step T0 in preparation for the next instruction's fetch, without any operator intervention.

Because the registers used throughout this design mix active-low and active-high clear conventions, the reset module must also produce both polarities of the reset signal from a single physical event, distributing an active-low reset (RESETL) to the modules that expect one and an active-high reset (RESETH) to those that expect the other.

Debounced Reset Generation: RESETL and RESETH

The manually triggered portion of the reset circuit is built around a single push-button, pulled to ground through a 10 k Ω resistor when not pressed and connected to the supply rail when pressed. Rather than feeding this switch directly into every register's clear pin, a small chain of 74HC00 NAND gates is used to both buffer the signal and generate both required polarities.

The first gate in the chain, U30:B, has both of its inputs tied together to the button's node, so that it functions as an inverting buffer rather than performing any logical combination; its output is a clean, buffered reset signal. This buffered signal is then passed through a second NAND gate, U30:C, whose two inputs are likewise tied together, inverting it a second time to recover the original polarity while providing an additional stage of buffering and drive. Between these two gates, the signal is made available at two points, taken directly as RESETH and, after the second inversion, as RESETL, so that the two complementary reset polarities required elsewhere in the design are both derived from the same physical button press without any risk of the two versions becoming desynchronized.

Step Counter Auto-Reset

The second function of this module, resetting the step counter automatically at the end of every instruction cycle, is implemented using a 74HC138 3-to-8 line decoder (U44), whose three select

inputs are driven directly by the three output bits of the step counter (U43). This decoder converts the counter's binary count into eight individually active-low outputs, one of which is active at any given time, corresponding to the counter's current value, functioning as an alternative to the shift-register-based ring counter used in some SAP-1 implementations, while producing an equivalent set of one-hot timing signals.

Since the instruction set implemented in this design uses only five steps (T0 through T4) per instruction, the decoder output corresponding to the next count value beyond the last step actually used is fed into a further pair of NAND gates (U30:D and U42:A), combined there with the buffered manual reset signal from the debounce stage described above. The output of this final gate is routed back to the step counter's own asynchronous master-reset input, so that the counter is cleared to zero either when the operator presses the reset button or automatically, without any operator action, as soon as the counter reaches the step value immediately following the last step used by any instruction. This produces a fixed five-step instruction cycle: every instruction, regardless of how many of its steps actually assert any control signals, causes the step counter to count from T0 through T4 and then reset, ready for the next instruction's fetch cycle.

Distribution to the Rest of the System

The RESETL and RESETH signals generated by the debounce stage are distributed throughout the rest of the machine, described individually in the sections covering each module, to the clear or master-reset inputs of the program counter, A and B registers, instruction register, memory address register, output register, and flags register, with the specific polarity used at each destination determined by whether that register's own clear input is active-high or active-low. This allows a single push-button to reset the entire machine to a known, consistent state in one operation, while the separate, automatically generated step-counter reset ensures that the instruction cycle continues to advance correctly between instructions without requiring any equivalent manual intervention during normal operation.

11 Instruction Set and Example Programs

11.1 The Extended Instruction Set

Building on the microcode architecture described in the previous section, the machine implements an instruction set of eleven mnemonics, extending Malvino's original five-instruction SAP-1 set (LDA, ADD, SUB, OUT, HLT) with six additional instructions: STA, LDI, JMP, JC, JZ, and an explicit NOP. Each instruction is encoded as a single byte, with the four most significant bits forming the opcode and the four least significant bits forming an operand, an address, an immediate value, or a jump target, depending on the instruction.

Table 3: Extended Instruction Set

Mnemonic	Opcode	Operand	Operation
NOP	0000	(unused)	No operation; fetch cycle only
LDA	0001	Address	Load A from the addressed RAM location
ADD	0010	Address	$A \leftarrow A + \text{RAM}[\text{address}]$; updates flags
SUB	0011	Address	$A \leftarrow A - \text{RAM}[\text{address}]$; updates flags
STA	0100	Address	Store A into the addressed RAM location
LDI	0101	Value	Load A directly with a 4-bit immediate value (0–15)
JMP	0110	Address	Unconditional jump to the specified address
JC	0111	Address	Jump to the specified address only if the carry flag is set
JZ	1000	Address	Jump to the specified address only if the zero flag is set
OUT	1110	(unused)	Display the contents of A
HLT	1111	(unused)	Stop the system clock

Opcodes 1001 through 1101 are unassigned in the current design; the microcode ROM is programmed with a fetch-only, no-operation control word for each of these values, so that any unintended or unprogrammed opcode encountered during execution behaves safely as a NOP rather than producing undefined behavior.

Every instruction occupies a fixed five-step instruction cycle (T_0 – T_4) regardless of how many steps its execution actually requires, as described in the control unit section; unused trailing steps assert no control signals and simply consume clock cycles without effect.

11.2 Program 1: Addition of Two Values

The simplest arithmetic program loads a value from memory, adds a second value, and displays the result.

This program exercises the fetch cycle, the LDA and ADD execution routines, the B register and ALU, and the output path, and served as one of the earliest validation programs used during the debugging process described in a later section.

Table 4: Program 1 – Addition

Address	Instruction	Machine Code
0	LDA 14	0001 1110
1	ADD 15	0010 1111
2	OUT	1110 0000
3	HLT	1111 0000
14	(operand 1)	e.g. 0000 1100 (12)
15	(operand 2)	e.g. 0000 0111 (7)

11.3 Program 2: Multiplication by Repeated Addition

Because the base instruction set has no dedicated multiply instruction, multiplication is instead implemented as a short loop that adds the multiplicand to a running product once for every unit of the multiplier, using JC to detect when the multiplier has been fully counted down.

Table 5: Program 2 – Multiplication

Address	Instruction	Machine Code
0	LDA 14	0001 1110
1	SUB 12	0011 1100
2	STA 14	0100 1110
3	JC 7	0111 0111
4	LDA 13	0001 1101
5	OUT	1110 0000
6	HLT	1111 0000
7	LDA 13	0001 1101
8	ADD 15	0010 1111
9	STA 13	0100 1101
10	JMP 0	0110 0000
12	constant 1	0000 0001
13	product (init 0)	0000 0000
14	multiplicand (x)	e.g. 0000 0110 (6)
15	multiplier (y)	e.g. 0000 0111 (7)

On each pass, 1 is subtracted from the running copy of x stored at address 14; because subtracting 1 from any nonzero value leaves the carry flag set, while subtracting 1 from exactly zero causes the result to wrap and clears the carry flag, JC correctly distinguishes “count not yet exhausted” from “count exhausted” using only the carry flag, without requiring a separate zero-detection instruction. The program terminates with the product of x and y in the A register and on the display.

11.4 Program 3: Division by Repeated Subtraction

Integer division of two exactly-divisible values is implemented with the same repeated-operation structure, subtracting the divisor from a running remainder and incrementing a quotient counter once per successful subtraction.

This program is limited to cases where the dividend is an exact multiple of the divisor, since the loop’s termination condition assumes the remainder eventually reaches exactly zero; a nonzero final remainder would leave the loop running indefinitely.

Table 6: Program 3 – Division

Address	Instruction	Machine Code
0	LDA 14	0001 1110
1	SUB 15	0011 1111
2	STA 14	0100 1110
3	JC 7	0111 0111
4	LDA 13	0001 1101
5	OUT	1110 0000
6	HLT	1111 0000
7	LDA 13	0001 1101
8	ADD 12	0010 1100
9	STA 13	0100 1101
10	JMP 0	0110 0000
12	constant 1	0000 0001
13	quotient (init 0)	0000 0000
14	dividend	e.g. 0000 0100 (4)
15	divisor	e.g. 0000 0010 (2)

11.5 Program 4: Data-Dependent Looping (Up/Down Counter)

The final example demonstrates the use of both conditional-jump instructions in a single program, alternating between counting up until an addition overflows and counting down until a subtraction reaches zero, continuing indefinitely.

Table 7: Program 4 – Up/Down Counter

Address	Instruction	Machine Code
0	OUT	1110 0000
1	ADD 15	0010 1111
2	JC 4	0111 0100
3	JMP 0	0110 0000
4	SUB 15	0011 1111
5	OUT	1110 0000
6	JZ 0	1000 0000
7	JMP 4	0110 0100
15	increment	0000 0001

Starting from zero, the A register is incremented and displayed on every pass until an addition overflows 8 bits, at which point JC redirects execution into a second loop that decrements and displays A until it reaches exactly zero, at which point JZ redirects execution back to the first loop. This program was used specifically to validate the correctness of both flag-detection paths together, since it is the first example in this report requiring both JC and JZ to function correctly within the same run.

11.6 Significance

None of Programs 2 through 4 can be expressed using Malvino’s original five-instruction SAP-1 set, since each depends on a jump whose target is determined by a previously computed result. Their successful execution demonstrates, concretely, the Turing-completeness argument introduced

earlier in this report: the addition of a single class of data-dependent control flow, implemented here as the JC and JZ instructions, is sufficient to extend the base architecture from a fixed-sequence calculator into a machine capable of expressing arbitrary loops and, by extension, any algorithm expressible with unbounded memory and time.

11.7 Conclusion

Now ,I have completely introduced the design, and by watching the project video you would see the programs running and see how it actually work. This experience provides a strong foundation for further study of real-world microprocessors and microcontrollers, where the same fundamental concepts are implemented in more advanced and highly integrated systems.

I have Finished

References

- [1] B. Eater, “Building an 8-bit breadboard computer.” https://youtu.be/HyznrdDSSGM?si=hvak4s0GfvdPM_t, 2015. YouTube Video Series. Accessed: 2026-08-20.
- [2] A. P. Malvino and J. A. Brown, *Digital Computer Electronics*. Career Education / McGraw-Hill, 3rd ed., 1992.
- [3] Texas Instruments, “Ne555 – precision timers, datasheet.” <https://www.ti.com/lit/ds/symlink/ne555.pdf>, 2015.
- [4] Texas Instruments, “Sn74hc173 – 4-bit d-type registers with 3-state outputs, datasheet.” <https://www.ti.com/lit/ds/symlink/cd54hc173.pdf?ts=1787158165292>, 2022.
- [5] Texas Instruments, “Sn74hc245 – octal bus transceivers with 3-state outputs, datasheet.” <https://www.ti.com/lit/ds/symlink/sn74hc245.pdf>, 2022.
- [6] Texas Instruments, “Sn74hc283 – 4-bit binary full adders with fast carry, datasheet.” <https://www.ti.com/lit/ds/symlink/cd74hc283.pdf?ts=1787226105449>, 2021.
- [7] Texas Instruments, “Sn74hc161 – 4-bit synchronous binary counters, datasheet.” <https://www.ti.com/lit/ds/symlink/sn74hc161.pdf>, 2021.
- [8] Texas Instruments, “Sn74ls189 – 64-bit random access memory, datasheet.” <https://www.datasheetarchive.com/datasheet/SN74LS189/Motorola?term=sn74ls189&version=1>, 1988.
- [9] Texas Instruments, “Sn74hc157 – quadruple 2-line to 1-line data selectors/multiplexers, datasheet.” <https://www.ti.com/lit/ds/symlink/sn74hc157.pdf>, 2021.
- [10] Microchip Technology, “27c256 – 256k (32k x 8) cmos eprom, datasheet.” <https://rocelec.widen.net/view/pdf/p6ig9bhsmu/MCHPS04540-1.pdf?t.download=true&u=5oefqw>, 2015.

Project Repository

you can access the project video via this link <https://drive.google.com/file/d/143fXrilSFHtZDMxH780QCFSpKUpq4RgY/view?usp=sharing>

you can access the project Repository via this link
<https://github.com/ahmedmuhammadkaram-bot/Simulation-of-SAP-1-8-bit-computer.git>